

Анализ программного и информационного обеспечения

Общая характеристика:

Сайт реализован на базе CMS Grav (ядро находится в папке grav). Это плоская файловая система управления контентом на PHP, использующая для хранения данных и настроек файлы в форматах YAML и Markdown, а не реляционную базу данных. Развертывание предполагается через Docker-контейнер.

Информационное обеспечение:

Представлено совокупностью YAML-файлов, текстовых файлов Markdown и JSON-файлов для интеграции.

1. **Конфигурации системы (user/config/):** YAML-файлы определяют все аспекты работы системы (system.yaml, site.yaml), безопасности (security.yaml, groups.yaml), медиа (media.yaml) и расписания заданий (scheduler.yaml).
2. **Пользовательские данные (user/accounts/):** Учетные записи администраторов хранятся в YAML-файлах (например, admin.yaml), содержащих хеши паролей, email и права доступа.
3. **Контент и данные (user/data/):** Хранятся неструктурированные данные, комментарии, логи ошибок (logerrors), уведомления и, предположительно, файлы резервных копий (backups.yaml).
4. **Темы и плагины (user/themes/, user/plugins/):** Содержат логику расширения функционала и отображения в виде PHP, Twig-шаблонов, YAML и CSS/JS.
5. **Страницы (user/pages/):** Контент страниц сайта хранится в папках, где каждая страница — это файл Markdown с метаданными.

Фрагмент руководства системного программиста

ГОСТ 19.503-79. ЕСПД. Руководство системного программиста

2. СТРУКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1. Общая архитектура и среда функционирования

Программное обеспечение сайта кафедры функционирует в среде веб-сервера под управлением ОС Linux в контейнере Docker. Сборка контейнера описана в файле Dockerfile. Непосредственное исполнение PHP-кода обеспечивается веб-сервером (Apache/Nginx), конфигурация взаимодействия с которым задается в файле .htaccess, расположенном в корне пользовательской части user/.

2.2. Структурные компоненты и их назначение

Структура ПО представляет собой типовую иерархию файловой CMS Grav, состоящую из следующих основных каталогов и файлов в корне проекта:

- Dockerfile — содержит инструкции для сборки Docker-образа программного комплекса.
- .gitignore / .gitlab-ci.yml / deploy.yml — служебные файлы системы контроля версий Git, непрерывной интеграции (GitLab CI) и деплоя.
- grav/ — корневой каталог ядра CMS Grav и пользовательских данных.

Детальная структура каталога grav/user/, как основного объекта сопровождения:

- config/ — **Модуль конфигурации системы.** Содержит набор YAML-файлов для управления всеми аспектами работы сайта.
 - system.yaml — основная конфигурация ядра CMS.
 - site.yaml — общие настройки сайта (название, автор).
 - security.yaml — правила и ограничения безопасности.
- accounts/ — **Модуль управления пользователями.** Содержит YAML-файлы учетных записей (admin.yaml), определяющих права доступа к системе.
- pages/ — **Модуль контента.** Хранит все страницы сайта в виде структурированных папок с файлами формата Markdown.
- data/ — **Модуль хранения данных времени выполнения.** Используется для логирования ошибок (logerrors), очередей уведомлений, фидов и других динамических данных, не относящихся к контенту страниц.
- plugins/ — **Модуль расширений.** Содержит устанавливаемые плагины, расширяющие функционал ядра.
- themes/ — **Модуль тем оформления.** Содержит файлы активной и резервных тем (шаблоны Twig, CSS, JS).
- **Файлы обратных вызовов:** oauth2callback.php, pr_gca_save_callback.php, pr_gd_save_callback.php и др. — являются скриптами-обработчиками для интеграции с Google API (авторизация OAuth2 и сохранение данных из Google Calendar, Google Drive и Google Data APIs), что следует из их именования и наличия конфигурационного файла client_id.json.

2.3. Взаимодействие компонентов

Системный программист должен учитывать следующую логику взаимодействия:

1. Все HTTP-запросы к сайту перенаправляются веб-сервером согласно правилам .htaccess в ядро Grav.
2. Ядро инициализирует окружение на основе файлов конфигурации из user/config/ (в первую очередь system.yaml).

3. При обращении к странице ядро считывает соответствующий Markdown-файл из `user/pages/` и обрабатывает его с помощью активной темы из `user/themes/` и активных плагинов из `user/plugins/`.
4. Специфические скрипты обратных вызовов (например, `oauth2callback.php`) работают автономно от основного ядра Grav, напрямую обрабатывая GET-запросы от внешних API Google для обновления данных, хранящихся в `user/data/`.

Список используемого программного обеспечения и технологий

1. Базовое системное ПО и среда исполнения

- **Docker:** Контейнеризация. Используется для упаковки приложения со всеми зависимостями в изолированный контейнер, обеспечивая идентичность среды на тестовом и продуктивном серверах (Dockerfile).
- **Linux (дистрибутив не указан):** Операционная система внутри контейнера и на целевом сервере.
- **Apache/Nginx (Веб-сервер):** Обслуживание HTTP-запросов. Взаимодействует с PHP-бэкендом. Правила перенаправления URL заданы в файле `.htaccess`, что типично для Apache.

2. Язык программирования и CMS

- **PHP (версия не указана в архиве):** Серверный скриптовый язык, на котором написана CMS Grav и дополнительные скрипты интеграции.
- **Grav (файловая CMS):** Ядро системы управления контентом. Не требует базы данных, хранит информацию в файлах Markdown и YAML. Версия и плагины управляются через файл `user/config/versions.yaml`.

3. Технологии хранения и описания данных

- **YAML:** Язык разметки для файлов конфигурации. Используется для описания всех настроек системы, параметров страниц и учетных записей администраторов.
- **Markdown (Extra):** Язык разметки контента. На нем пишутся все текстовые страницы сайта, обрабатываемые ядром Grav. В настройках включен расширенный синтаксис (`pages.markdown.extra: true`).
- **JSON:** Формат обмена данными. Используется для хранения секретного ключа интеграции с Google API (`client_id.json`).

4. Технологии фронтенда и шаблонизации

- **Twig:** Шаблонизатор PHP. Используется темами и плагинами Grav для генерации HTML. Также может обрабатываться внутри контента страниц (включена опция `pages.process.twig: true`).

- **CSS / JavaScript (предположительно):** Содержатся в файлах тем (user/themes/). Управляются через систему ассетов Grav (assets в конфиге), которая отвечает за минификацию и конвейеризацию.

5. Технологии интеграции и API

- **Google OAuth 2.0:** Протокол авторизации. Используется для получения безопасного доступа к аккаунтам Google API (наличие файлов oauth2callback.php и client_id.json).
- **Google APIs (Drive, Calendar, Data):** Внешние облачные сервисы. Интеграция с ними реализована через специализированные скрипты обратного вызова (pr_gd_save_callback.php для Google Drive, pr_gca_save_callback.php для Google Calendar, pr_gda_save_callback.php для Google Data).

6. Технологии разработки и развертывания (DevOps)

- **Git:** Система контроля версий (файлы .gitignore, .gitkeep).
- **GitLab CI:** Система непрерывной интеграции. Конфигурация пайплайнов для автоматической сборки, тестирования и деплоя описана в файле .gitlab-ci.yml.
- **YAML (Deploy):** Описание процесса развертывания (deployment) в файле deploy.yaml.

7. Дополнительные серверные сервисы (опционально)

- **Redis / Memcache(d):** Драйверы для кеширования. Сервер и порт для подключения (cache.redis, cache.memcache, cache.memcached) сконфигурированы в system.yaml, но не заполнены (по умолчанию используется файловый кеш). Могут быть задействованы для ускорения работы при росте нагрузки.